

Module : NoSQL Security

Séance 1 — Découvrir NoSQL & Prise en main de MongoDB

Rapport de Travaux Pratiques

Filière 2^{ème} année Cybersécurité
Date 27 février 2026
Outils Docker, MongoDB 7, mongosh
Base noseclab / security_logs

Contents

1	Introduction	2
1.1	Contexte du TP	2
1.2	Objectifs pédagogiques	2
1.3	Environnement de travail	3
2	Mini-TP 3 — Démarrage Docker & MongoDB	4
2.1	Objectif	4
2.2	Connexion au conteneur	4
2.2.1	Analyse de la capture	4
3	Mini-TP 4 — Création de la base & insertion du dataset	6
3.1	Objectif	6
3.2	Insertion des documents avec insertMany	6
3.3	Vérification du dataset	7
3.3.1	Analyse des résultats	8
4	Mini-TP 5 — Opérations CRUD	9
4.1	Exercice B — Read : Requêtes de lecture avec filtres	9
4.1.1	B2 — Logs de l'utilisateur bob	9
4.1.2	B3 — Logs de sévérité critical	11
4.1.3	B4 — Logs provenant de Chine (champ imbriqué)	12
4.2	Exercice C — Update : Modification de documents	13
4.2.1	C1 — updateOne : marquage d'un faux positif	13
4.2.2	C2 — updateMany : politique de sévérité globale	13
4.3	Exercice D — Delete : Suppression d'un document	14
5	Bilan & Conclusion	16
5.1	Récapitulatif des opérations réalisées	16
5.2	Points de sécurité identifiés	16
5.3	Conclusion	17

1. Introduction

1.1 Contexte du TP

Ce travail pratique constitue la première séance du module **NoSQL Security**, destiné aux étudiants de 2^{ème} année en Cybersécurité. L'objectif est de découvrir les bases de données NoSQL, en particulier **MongoDB**, dans un contexte orienté sécurité des systèmes d'information.

Les bases de données NoSQL sont aujourd'hui omniprésentes dans les architectures modernes : plateformes cloud, systèmes de détection d'intrusion (IDS), SIEM (*Security Information and Event Management*) et applications à forte volumétrie. Comprendre leur fonctionnement, leurs avantages et surtout leurs risques de sécurité est devenu une compétence indispensable pour tout professionnel de la cybersécurité.

1.2 Objectifs pédagogiques

À l'issue de cette séance, l'étudiant doit être capable de :

- Expliquer les différences fondamentales entre SQL et NoSQL, et citer des cas d'usage pertinents pour chaque paradigme.
- Comprendre le modèle Document de MongoDB : base, collection, document, champ, `_id`.
- Lancer un serveur MongoDB via **Docker** et s'y connecter avec **mongosh**.
- Réaliser l'ensemble des opérations **CRUD** (Create, Read, Update, Delete) sur une collection de logs de sécurité.
- Formuler des requêtes avancées avec filtres, tri, limite et projection.
- Identifier les failles de configuration par défaut de MongoDB (port exposé, absence d'authentification, *bind address*).

1.3 Environnement de travail

Le TP est réalisé sur une machine Windows avec Docker Desktop. Le conteneur MongoDB est lancé avec le mapping de port `-p 27017:27017` et un volume persistant. Toutes les commandes sont exécutées depuis **PowerShell** via `docker exec -it mongo-noseclab mongosh`.

Le jeu de données utilisé, `security_logs`, simule un mini-SIEM contenant 12 documents représentant des événements de sécurité variés : tentatives de connexion, accès fichiers, scans de ports et escalades de privilèges.

2. Mini-TP 3 — Démarrage Docker & MongoDB

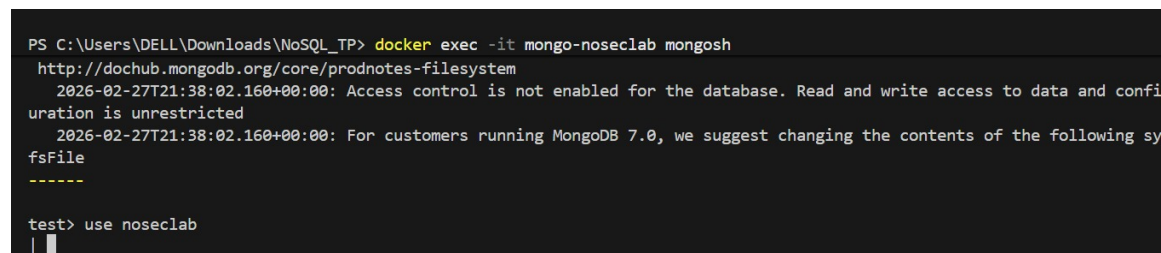
2.1 Objectif

Lancer un conteneur MongoDB et se connecter via **mongosh**. Ce mini-TP valide la compétence C3 : lancer MongoDB via Docker et obtenir le prompt interactif.

2.2 Connexion au conteneur

La commande suivante permet d'entrer dans le conteneur MongoDB et de lancer le shell interactif **mongosh** :

```
docker exec -it mongo-noseclab mongosh
```



```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo-noseclab mongosh
http://dochub.mongodb.org/core/prodnotes-filesystem
2026-02-27T21:38:02.160+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2026-02-27T21:38:02.160+00:00: For customers running MongoDB 7.0, we suggest changing the contents of the following system file
fsFile
-----
test> use noseclab
|
```

Figure 2.1: Connexion à mongosh via Docker — le prompt **test>** confirme la connexion réussie

2.2.1 Analyse de la capture

Plusieurs informations importantes sont visibles au démarrage :

- Un avertissement **Access control is not enabled** indique que l'authentification MongoDB est désactivée par défaut — tout utilisateur ayant accès au port 27017 dispose d'un accès complet en lecture et écriture.
- MongoDB 7.0 recommande de modifier certains paramètres système (*fsFile*) pour optimiser les performances.

- La commande `use nosqlab` permet ensuite de basculer vers la base de données du TP.

Observation sécurité

L'avertissement `Access control is not enabled` est un signal critique. En environnement de production, cette configuration expose l'intégralité des données à quiconque peut joindre le port 27017. C'est ce qui a causé les attaques ransomware MongoDB de 2017.

3. Mini-TP 4 — Création de la base & insertion du dataset

3.1 Objectif

Créer la base `noseclab`, insérer le dataset `security_logs` (12 documents), puis vérifier son contenu. Ce mini-TP valide la compétence C4 (Create).

3.2 Insertion des documents avec `insertMany`

Après avoir basculé vers la base `noseclab`, la commande `insertMany` permet d'insérer en une seule opération l'ensemble du dataset :

```
use noseclab
db.security_logs.insertMany([
  {
    timestamp: ISODate("2026-02-25T08:01:00Z"),
    src_ip: "192.168.1.10",
    user: "alice",
    action: "login",
    status: "success",
    device: "laptop",
    geo: { country: "FR", city: "Paris" },
    severity: "info"
  },
  // ... 11 autres documents
])
```

```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo-noseclab mongosh
test> use noseclab
|
switched to db noseclab
noseclab> db.security_logs.insertMany([
|   {
|     timestamp: ISODate("2026-02-25T08:01:00Z"),
|     src_ip: "192.168.1.10",
|     user: "alice",
|     action: "login",
|   }
| ])
```

Figure 3.1: Début de l'insertion du dataset security_logs avec insertMany

3.3 Vérification du dataset

Après insertion, les commandes de vérification confirment l'état de la base :

```
show dbs          -- Liste toutes les bases
show collections  -- Liste les collections de noseclab
db.security_logs.countDocuments() -- Compte les documents
db.security_logs.findOne()      -- Affiche le premier document
```



```
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo-noseclab mongosh
'8': ObjectId('69a20f9ccf36e658da8563b9'),
'9': ObjectId('69a20f9ccf36e658da8563ba'),
'10': ObjectId('69a20f9ccf36e658da8563bb'),
'11': ObjectId('69a20f9ccf36e658da8563bc')
}
}
noseclab> show dbs
|
admin      40.00 KiB
config     12.00 KiB
local      40.00 KiB
noseclab   40.00 KiB
noseclab> show collections
|
security_logs
noseclab> db.security_logs.countDocuments()
|
12
noseclab> db.security_logs.findOne()
|
{
  _id: ObjectId('69a20f9ccf36e658da8563b1'),
  timestamp: ISODate('2026-02-25T08:01:00.000Z'),
  src_ip: '192.168.1.10',
  user: 'alice',
  action: 'login',
  status: 'success',
  device: 'laptop',
  geo: { country: 'FR', city: 'Paris' },
  severity: 'info'
}
noseclab> 
```

Figure 3.2: Vérification : 12 documents insérés, premier document affiché avec tous ses champs

3.3.1 Analyse des résultats

- `show dbs` confirme la création de la base `noseclab` (40 KiB) aux côtés des bases système (`admin`, `config`, `local`).
- `countDocuments()` retourne **12**, ce qui correspond au nombre de documents insérés.
- `findOne()` affiche le premier document avec son `_id` généré automatiquement (`ObjectId`), ses champs plats (`user`, `action`, `status`) et son objet imbriqué `geo`.

4. Mini-TP 5 — Opérations CRUD

4.1 Exercice B — Read : Requêtes de lecture avec filtres

4.1.1 B2 — Logs de l'utilisateur bob

La requête suivante filtre les documents par utilisateur :

```
db.security_logs.find({ user: "bob" })
```

```
noseclab> db.security_logs.find({ user: "bob" })
[
  {
    _id: ObjectId('69a20f9ccf36e658da8563b3'),
    timestamp: ISODate('2026-02-25T08:12:45.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'warning'
  },
  {
    _id: ObjectId('69a20f9ccf36e658da8563b4'),
    timestamp: ISODate('2026-02-25T08:13:10.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
    device: 'desktop',
    geo: { country: 'FR', city: 'Lyon' },
    severity: 'warning'
  },
  {
    _id: ObjectId('69a20f9ccf36e658da8563b5'),
    timestamp: ISODate('2026-02-25T08:13:42.000Z'),
    src_ip: '10.0.0.50',
    user: 'bob',
    action: 'login',
    status: 'failure',
  }
]
```

Figure 4.1: Résultats de la recherche filtrée sur `user: "bob"` — 3 tentatives de connexion échouées depuis Lyon

Analyse

Les résultats révèlent un comportement suspect : l'utilisateur **bob** a effectué **trois tentatives de connexion échouées** consécutives depuis la même IP (10.0.0.50), en l'espace de quelques minutes. Ce pattern correspond à une **attaque brute-force** ou à un compte compromis. La sévérité **warning** est justifiée.

4.1.2 B3 — Logs de sévérité critical

```
db.security_logs.find({ severity: "critical" })
```

```
noseclab> db.security_logs.find({ severity: "critical" })
|
[
  {
    _id: ObjectId('69a20f9ccf36e658da8563b7'),
    timestamp: ISODate('2026-02-25T09:00:00.000Z'),
    src_ip: '172.16.0.99',
    action: 'port_scan',
    status: 'detected',
    severity: 'critical'
  },
  {
    _id: ObjectId('69a20f9ccf36e658da8563bb'),
    timestamp: ISODate('2026-02-25T10:30:00.000Z'),
    src_ip: '192.168.1.25',
    user: 'charlie',
    action: 'privilege_escalation',
    status: 'failure',
    device: 'server-web01',
    geo: { country: 'FR', city: 'Marseille' },
    severity: 'critical'
  }
]
noseclab> db.security_logs.find({ "geo.country": "CN" })
|
[
  {
    _id: ObjectId('69a20f9ccf36e658da8563b9'),
    timestamp: ISODate('2026-02-25T10:00:00.000Z'),
    src_ip: '203.0.113.42',
    action: 'login',
    status: 'failure',
```

Figure 4.2: Documents de sévérité critical : scan de ports et tentative d'escalade de privilèges

Analyse

Deux événements critiques sont identifiés :

- Un **scan de ports** (port_scan) depuis l'IP interne 172.16.0.99, détecté à 09h00.

- Une tentative d'**escalade de privilèges** (`privilege_escalation`) par l'utilisateur `charlie` sur le serveur `server-web01`, depuis Marseille.

4.1.3 B4 — Logs provenant de Chine (champ imbriqué)

```
db.security_logs.find({ "geo.country": "CN" })
```

```
]
noseclab> db.security_logs.find({ "geo.country": "CN" })
|
[
  {
    _id: ObjectId('69a20f9ccf36e658da8563b9'),
    timestamp: ISODate('2026-02-25T10:00:00.000Z'),
    src_ip: '203.0.113.42',
    action: 'login',
    status: 'failure',
    geo: { country: 'CN', city: 'Beijing' },
    severity: 'high'
  },
  {
    _id: ObjectId('69a20f9ccf36e658da8563ba'),
    timestamp: ISODate('2026-02-25T10:01:00.000Z'),
    src_ip: '203.0.113.42',
    action: 'login',
    status: 'failure',
    geo: { country: 'CN', city: 'Beijing' },
    severity: 'high'
  }
]
noseclab> 
```

Figure 4.3: Requête sur champ imbriqué `geo.country`: "CN" — deux tentatives depuis Beijing

Analyse

La notation pointée ("`geo.country`") permet d'accéder aux champs d'objets imbriqués dans MongoDB. Les deux résultats montrent des tentatives de connexion échouées depuis l'IP 203.0.113.42 (Beijing), avec une sévérité `high`. Ce type de connexion depuis un pays étranger non attendu est un indicateur de compromission (IoC) classique.

4.2 Exercice C — Update : Modification de documents

4.2.1 C1 — updateOne : marquage d'un faux positif

L'analyse a conclu que le scan de ports (172.16.0.99) est un faux positif. On ajoute le champ `investigated: true` et on abaisse la sévérité à "info" :

```
db.security_logs.updateOne(
  { src_ip: "172.16.0.99", action: "port_scan" },
  { $set: { investigated: true, severity: "info" } }
)
```

```
noseclab> db.security_logs.updateOne(
|   { src_ip: "172.16.0.99", action: "port_scan" },
|   { $set: { investigated: true, severity: "info" } }
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
noseclab> 
```

Figure 4.4: `updateOne` sur le document `port_scan` : `matchedCount: 1`, `modifiedCount: 1`

4.2.2 C2 — updateMany : politique de sévérité globale

On renforce la politique de sécurité en classifiant tous les `login failure` en sévérité `high` :

```
db.security_logs.updateMany(
  { action: "login", status: "failure" },
  { $set: { severity: "high" } }
)
```

```
noseclab> db.security_logs.updateOne(
|   { src_ip: "172.16.0.99", action: "port_scan" },
|   { $set: { investigated: true, severity: "info" } }
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
noseclab> db.security_logs.updateMany(
|   { action: "login", status: "failure" },
|   { $set: { severity: "high" } }
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 6,
|   modifiedCount: 3,
|   upsertedCount: 0
| }
```

Figure 4.5: updateMany : matchedCount: 6, modifiedCount: 3 — mise à jour de masse réussie

Analyse

Le résultat de updateMany montre :

- matchedCount: 6 — 6 documents correspondent au filtre action: "login", status: "failure".
- modifiedCount: 3 — seulement 3 ont été effectivement modifiés (les 3 autres avaient déjà la sévérité "high").

Cela illustre le comportement idempotent de \$set : MongoDB ne modifie un document que si la valeur change réellement.

4.3 Exercice D — Delete : Suppression d'un document

Le log de test de l'utilisateur eve est supprimé :

```
db.security_logs.deleteOne({ user: "eve", src_ip: "198.51.100.7" })
```

```
noseclab> db.security_logs.deleteOne({ user: "eve", src_ip: "198.51.100.7" })
|
{ acknowledged: true, deletedCount: 1 }
noseclab> █
```

Figure 4.6: `deleteOne` : `deletedCount`: 1 — suppression confirmée

La suppression est confirmée par `deletedCount`: 1. Un `countDocuments()` après cette opération retourne **12**, ce qui est cohérent avec les attentes du TP.

5. Bilan & Conclusion

5.1 Récapitulatif des opérations réalisées

Opération	Commande utilisée	Résultat
Connexion Docker	<code>docker exec -it mongo-noseclab mongosh</code>	Succès
Insertion dataset	<code>insertMany([...])</code>	12 documents
Lecture filtrée	<code>find({ user: "bob" })</code>	3 docs
Filtre sévérité	<code>find({ severity: "critical" })</code>	2 docs
Filtre imbriqué	<code>find({ "geo.country": "CN" })</code>	2 docs
Mise à jour unique	<code>updateOne(..., \$set)</code>	1 modifié
Mise à jour masse	<code>updateMany(..., \$set)</code>	3 modifiés
Suppression	<code>deleteOne({ user: "eve" })</code>	1 supprimé

Table 5.1: Récapitulatif des opérations CRUD réalisées

5.2 Points de sécurité identifiés

Ce TP met en évidence plusieurs risques de sécurité inhérents à une configuration MongoDB par défaut :

1. **Absence d'authentification** : MongoDB démarre sans contrôle d'accès. Tout utilisateur joignant le port 27017 dispose d'un accès total.
2. **Port 27017 exposé** : le mapping `-p 27017:27017` rend le serveur accessible depuis le réseau hôte, voire depuis l'extérieur selon la configuration du pare-feu.

3. **Bind address 0.0.0.0** : par défaut, MongoDB écoute sur toutes les interfaces réseau.
4. **Données sensibles sans projection** : une API qui interroge MongoDB sans projection peut exposer des champs confidentiels (IP internes, tokens, identifiants).

5.3 Conclusion

Cette première séance a permis de mettre en pratique l'ensemble du cycle CRUD sur une base MongoDB orientée logs de sécurité. La manipulation du dataset `security_logs` illustre concrètement comment un analyste SOC pourrait interroger, mettre à jour et gérer des événements de sécurité dans une base NoSQL. Les opérations de filtrage par utilisateur, par sévérité et par géolocalisation (champs imbriqués) sont des briques fondamentales pour la corrélation d'événements en cybersécurité.

La prochaine séance approfondira la sécurisation de MongoDB : activation de l'authentification, gestion des rôles RBAC, chiffrement des données et surveillance des accès.